

OLIST E-COMMERCE DATA ANALYSIS

Python-Based Exploratory Data Analysis

Beyzanur Seçgin
2026

Introduction

This project focuses on analyzing an e-commerce dataset using Python. The main purpose of the project is to understand the company's sales performance, customer behavior, product categories, sellers, and delivery performance through data analysis.

The dataset contains information about orders, customers, products, sellers, prices, order statuses, and delivery dates. By analyzing these data, I aimed to identify important patterns and answer different business questions.

In this project, I first explored and prepared the data, and then performed several analyses using Python. The results were interpreted through key findings to understand the business performance and identify important insights from the data.

First step is putting the data in python.

```
orders = pd.read_csv("D:/1/ME/portfolio  
(1)/pörtfolyo.2026/001.README.md/archive/olist_orders_dataset.csv")
```

After putting the data. We will look for data. Now looking for what the data looks like.

Code:

```
print(orders.shape)
```

Output:

(99441, 8)

The dataset consists of 99,441 rows and 8 columns, representing customer order information.

Code:

```
print(orders.head)
```

looking for first five data

Output:

This function displays the first five rows of the dataset for an initial inspection.

Code:

```
print(orders.info())
```

looking for data have all data or not

Output:

#	Column	Non-Null Count	Dtype
0	order_id	99441 non-null	object
1	customer_id	99441 non-null	object
2	order_status	99441 non-null	object
3	order_purchase_timestamp	99441 non-null	object
4	order_approved_at	99281 non-null	object
5	order_delivered_carrier_date	97658 non-null	object
6	order_delivered_customer_date	96476 non-null	object
7	order_estimated_delivery_date	99441 non-null	object

The dataset has non-null values, it means we can use this data with no missing values.

Code:

```
print(orders.columns)
```

Giving us columns names

Output:

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',  
      'order_approved_at', 'order_delivered_carrier_date',  
      'order_delivered_customer_date', 'order_estimated_delivery_date'],  
      dtype='object')
```

The column names were examined to understand the structure of the dataset and identify the available variables for analysis.

Code:

```
print(orders.isnull().sum())
```

Controlling dataset have all or not

Output:

```
order_id                0  
customer_id            0  
order_status            0  
order_purchase_timestamp 0  
order_approved_at       160  
order_delivered_carrier_date 1783  
order_delivered_customer_date 2965  
order_estimated_delivery_date 0  
dtype: int64
```

Missing Value Analysis

The orders dataset contains missing values in three columns:

- order_approved_at: 160 missing values
- order_delivered_carrier_date: 1,783 missing values

order_delivered_customer_date: 2,965 missing values

These missing values are likely related to order processing and delivery status. For example, cancelled or undelivered orders may not have approval or delivery timestamps. Therefore, the missing values were retained for further analysis instead of being removed.

Code:

```
print(orders["order_status"].value_counts())
```

Using this code for missing values.

Output:

```
order_status
delivered      96478
shipped         1107
canceled         625
unavailable     609
invoiced         314
processing       301
created           5
approved         2
```

Order Status Analysis

The majority of orders (**96,478**) were successfully delivered, indicating a high order completion rate.

A smaller number of orders were still in progress (shipped, processing, approved, and created) or were marked as canceled and unavailable.

This explains why some delivery-related columns contain missing values. Since these missing values are associated with the order lifecycle rather than data quality issues, they were retained for further analysis.

Code:

```
print(orders.duplicated().sum())
```

Output:

0

No duplicate records were found in the orders dataset.

Code:

```
print(orders.describe())
```

Output:

Descriptive Statistics

The `describe()` function was used to summarize the categorical variables in the orders dataset.

The analysis showed that the dataset contains **8 unique order statuses**, with "**delivered**" being the most frequent status, appearing **96,478** times.

The output also confirmed that each order and customer identifier is unique, indicating that the dataset does not contain duplicate order records.

Now making same thing to customers dataset

All Codes:

```
customers = pd.read_csv(r"D:\1\ME\portfolio
(1)\pörtfolyo.2026\001.README.md\archive\olist_customers_dataset.csv")
print(customers.shape)
print(customers.head)
print(customers.info)
print(customers.columns)
print(customers.isnull().sum())
print(customers.duplicated().sum())
print(customers.describe())
<bound method NDFrame.head of
customer_unique_id  customer_zip_code_prefix  customer_id
customer_city
customer_state
0      06b8999e2fba1a1fbc88172c00ba8bc7  861eff4711a542e4b93843c6dd7febb0
14409      franca  SP
1      18955e83d337fd6b2def6b18a428ac77  290c77bc529b7ac935b93aa66c333dc3
9790  sao bernardo do campo  SP
2      4e7b3e00288586ebd08712fdd0374a03  060e732b5b29e8181a18229c7b0b2b5e
1151      sao paulo  SP
3      b2b6027bc5c5109e529d4dc6358b12c3  259dac757896d24d7702b9acbbff3f3c
8775      mogi das cruzeiros  SP
4      4f2d8ab171c80ec8364f7c12e35b23ad  345ecd01c38d18a9036ed96c73b8d066
13056      campinas  SP
...
...
...
99436  17ddf5dd5d51696bb3d7c6291687be6f  1a29b476fee25c95fbafc67c5ac95cf8
3937      sao paulo  SP
99437  e7b71a9017aa05c9a7fd292d714858e8  d52a67c98be1cf6a5c84435bd38d095d
6764      taboao da serra  SP
99438  5e28dfe12db7fb50a4b2f691faecea5e  e9f50caf99f032f0bf3c55141f019d99
60115      fortaleza  CE
99439  56b18e2166679b8a959d72dd06da27f9  73c2643a0a458b49f58cea58833b192e
92120      canoas  RS
99440  274fa6071e5e17fe303b9748641082c8  84732c5050c01db9b23e19ba39899398
6703      cotia  SP
```

```
[99441 rows x 5 columns]>
<bound method DataFrame.info of                                     customer_id
customer_unique_id  customer_zip_code_prefix  customer_city
customer_state
0      06b8999e2fba1a1fbc88172c00ba8bc7  861eff4711a542e4b93843c6dd7febb0
14409                                franca  SP
1      18955e83d337fd6b2def6b18a428ac77  290c77bc529b7ac935b93aa66c333dc3
9790  sao bernardo do campo  SP
2      4e7b3e00288586ebd08712fdd0374a03  060e732b5b29e8181a18229c7b0b2b5e
1151                                sao paulo  SP
3      b2b6027bc5c5109e529d4dc6358b12c3  259dac757896d24d7702b9acbbff3f3c
8775                                mogi das cruzeiros  SP
4      4f2d8ab171c80ec8364f7c12e35b23ad  345ecd01c38d18a9036ed96c73b8d066
13056                                campinas  SP
...
...
...
99436  17ddf5dd5d51696bb3d7c6291687be6f  1a29b476fee25c95fbafc67c5ac95cf8
3937                                sao paulo  SP
99437  e7b71a9017aa05c9a7fd292d714858e8  d52a67c98be1cf6a5c84435bd38d095d
6764                                taboao da serra  SP
99438  5e28dfe12db7fb50a4b2f691faeccea5e  e9f50caf99f032f0bf3c55141f019d99
60115                                fortaleza  CE
99439  56b18e2166679b8a959d72dd06da27f9  73c2643a0a458b49f58cea58833b192e
92120                                canoas  RS
99440  274fa6071e5e17fe303b9748641082c8  84732c5050c01db9b23e19ba39899398
6703                                cotia  SP
```

```
[99441 rows x 5 columns]>
Index(['customer_id', 'customer_unique_id', 'customer_zip_code_prefix',
      'customer_city', 'customer_state'],
      dtype='object')
```

```
customer_id      0
customer_unique_id  0
customer_zip_code_prefix  0
customer_city      0
customer_state    0
dtype: int64
0
customer_zip_code_prefix
count      99441.000000
mean       35137.474583
std        29797.938996
min         1003.000000
25%        11347.000000
50%        24416.000000
75%        58900.000000
max        99990.000000
```

Key Findings:

The customers dataset contains **99,441 records** and **5 variables**, providing customer identification and location information.

The initial inspection confirmed that the dataset contains customer identifiers and geographical information, including city and state.

The dataset contains complete customer information with no missing values. All categorical fields are stored as object data types, while the ZIP code is stored as an integer.

No missing values were detected in the customers dataset, indicating high data completeness.

No duplicate records were identified in the customers dataset

Although descriptive statistics were generated for the ZIP code, these values have limited business significance because ZIP codes are identifiers rather than measurable quantities.

```
merged = pd.merge(
    orders,
    customers,
    on="customer_id",
    how="inner"
)

print(merged.shape)
print(merged.head())

(99441, 12)
```

	order_id	customer_id
order_status ...	customer_zip_code_prefix	customer_city
customer_state		
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d
delivered ...	3149	sao paulo
SP		
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef
delivered ...	47813	barreiras
BA		
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089
delivered ...	75265	vianopolis
GO		
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82
delivered ...	59296	sao goncalo do amarante
RN		
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c
delivered ...	9195	santo andre
SP		

The orders and customers datasets were merged using the customer_id field. This integration combines order information with customer location details, enabling geographical and customer-based analysis.

Putting one more dataset

```
order_items = pd.read_csv(r"D:\1\ME\portfolio
(1)\pörtfolyo.2026\001.README.md\archive\olist_order_items_dataset.csv")
print(order_items.shape)
print(order_items.head)
print(order_items.info)
print(order_items.columns)
print(order_items.isnull().sum())
print(order_items.duplicated().sum())
print(order_items.describe())
```

Output:

```
value
0      00010242fe8c5a6d1ba2dd792cb16214      1
4244733e06e7ecb4970a6e2683c13e61  ...  2017-09-19 09:45:35   58.90
13.29
1      00018f77f2f0320c557190d7a144bdd3      1
e5f2d52b802189ee658865ca93d83a8f  ...  2017-05-03 11:05:13  239.90
19.93
2      000229ec398224ef6ca0657da4fc703e      1
c777355d18b72b67abbef9df44fd0fd  ...  2018-01-18 14:48:30  199.00
17.87
3      00024acbcdff0a6daa1e931b038114c75      1
7634da152a4610f1595efa32f14722fc  ...  2018-08-15 10:10:18   12.99
12.79
4      00042b26cf59d7ce69dfabb4e55b4fd9      1
ac6c3623068f30de03045865e4e10089  ...  2017-02-13 13:57:51  199.90
18.14
...
...
112645  fffc94f6ce00a00581880bf54a75a037      1
4aa6014eceb682077f9dc4bffeabc05b0  ...  2018-05-02 04:11:01  299.99
43.41
112646  fffc46ef2263f404302a634eb57f7eb      1
32e07fd915822b0765e448c4dd74c828  ...  2018-07-20 04:31:48  350.00
36.53
112647  fffce4705a9662cd70adb13d4a31832d      1
72a30483855e2eafc67aee5dc2560482  ...  2017-10-30 17:14:25   99.90
16.95
112648  fffe18544ffabc95dfada21779c9644f      1
9c422a519119dcad7575db5af1ba540e  ...  2017-08-21 00:04:32   55.99
8.72
```

```
112649  fffe41c64501cc87c801fd61db3f6244 1
350688d9dc1e75ff97be326363655e01 ... 2018-06-12 17:10:13 43.00
12.79
```

```
[112650 rows x 7 columns]>
Index(['order_id', 'order_item_id', 'product_id', 'seller_id',
      'shipping_limit_date', 'price', 'freight_value'],
      dtype='object')
order_id      0
order_item_id  0
product_id    0
seller_id     0
shipping_limit_date  0
price         0
freight_value  0
dtype: int64
0
```

	order_item_id	price	freight_value
count	112650.000000	112650.000000	112650.000000
mean	1.197834	120.653739	19.990320
std	0.705124	183.633928	15.806405
min	1.000000	0.850000	0.000000
25%	1.000000	39.900000	13.080000
50%	1.000000	74.990000	16.260000
75%	1.000000	134.900000	21.150000
max	21.000000	6735.000000	409.680000

Key Findings:

putting one more dataset:

```
products = pd.read_csv(r"D:\1\ME\portfolio
(1)\pörtfolyo.2026\001.README.md\archive\olist_products_dataset.csv")
print(products.shape)
print(products.head)
print(products.info)
print(products.columns)
print(products.isnull().sum())
print(products.duplicated().sum())
print(products.describe())
```

output:

```
(32951, 9)
<bound method NDFrame.head of                                     product_id
product_category_name  product_name_lenght  ...  product_length_cm
product_height_cm  product_width_cm
0      1e9e8ef04dbcff4541ed26657ea517e5      perfumaria
40.0  ...      16.0      10.0      14.0
1      3aa071139cb16b67ca9e5dea641aaa2f      artes
44.0  ...      30.0      18.0      20.0
2      96bd76ec8810374ed1b65e291975717f      esporte_lazer
46.0  ...      18.0      9.0      15.0
3      cef67bcfe19066a932b7673e239eb23d      bebes
27.0  ...      26.0      4.0      26.0
4      9dc1a7de274444849c219cff195d0b71      utilidades_domesticas
37.0  ...      20.0      17.0      13.0
...      ...      ...      ...
...      ...      ...      ...
32946  a0b7d5a992ccda646f2d34e418fff5a0      moveis_decoracao
45.0  ...      40.0      40.0      40.0
32947  bf4538d88321d0fd4412a93c974510e6  construcao_ferramentas_iluminacao
41.0  ...      16.0      19.0      16.0
32948  9a7c6041fa9592d9d9ef6cfe62a71f8c      cama_mesa_banho
50.0  ...      27.0      7.0      27.0
32949  83808703fc0706a22e264b9d75f04a2e      informatica_acessorios
60.0  ...      31.0      13.0      20.0
32950  106392145fca363410d287a815be6de4      cama_mesa_banho
58.0  ...      12.0      2.0      7.0

[32951 rows x 9 columns]>
<bound method DataFrame.info of                                     product_id
product_category_name  product_name_lenght  ...  product_length_cm
product_height_cm  product_width_cm
0      1e9e8ef04dbcff4541ed26657ea517e5      perfumaria
40.0  ...      16.0      10.0      14.0
1      3aa071139cb16b67ca9e5dea641aaa2f      artes
44.0  ...      30.0      18.0      20.0
2      96bd76ec8810374ed1b65e291975717f      esporte_lazer
46.0  ...      18.0      9.0      15.0
3      cef67bcfe19066a932b7673e239eb23d      bebes
27.0  ...      26.0      4.0      26.0
4      9dc1a7de274444849c219cff195d0b71      utilidades_domesticas
37.0  ...      20.0      17.0      13.0
...      ...      ...      ...
...      ...      ...      ...
32946  a0b7d5a992ccda646f2d34e418fff5a0      moveis_decoracao
45.0  ...      40.0      40.0      40.0
32947  bf4538d88321d0fd4412a93c974510e6  construcao_ferramentas_iluminacao
41.0  ...      16.0      19.0      16.0
```

```

32948  9a7c6041fa9592d9d9ef6cfe62a71f8c      cama_mesa_banho
50.0  ...                27.0                7.0                27.0
32949  83808703fc0706a22e264b9d75f04a2e      informatica_acessorios
60.0  ...                31.0                13.0                20.0
32950  106392145fca363410d287a815be6de4      cama_mesa_banho
58.0  ...                12.0                2.0                7.0

[32951 rows x 9 columns]>
Index(['product_id', 'product_category_name', 'product_name_lenght',
      'product_description_lenght', 'product_photos_qty', 'product_weight_g',
      'product_length_cm', 'product_height_cm', 'product_width_cm'],
      dtype='object')
product_id                0
product_category_name      610
product_name_lenght        610
product_description_lenght  610
product_photos_qty         610
product_weight_g           2
product_length_cm          2
product_height_cm          2
product_width_cm           2
dtype: int64
0
      product_name_lenght  product_description_lenght  product_photos_qty
product_weight_g  product_length_cm  product_height_cm  product_width_cm
count          32341.000000          32341.000000          32341.000000
32949.000000          32949.000000          32949.000000          32949.000000
mean             48.476949             771.495285             2.188986
2276.472488          30.815078             16.937661             23.196728
std              10.245741             635.115225             1.736766
4282.038731          16.914458             13.637554             12.079047
min              5.000000              4.000000             1.000000
0.000000           7.000000              2.000000             6.000000
25%              42.000000             339.000000             1.000000
300.000000          18.000000              8.000000            15.000000
50%              51.000000             595.000000             1.000000
700.000000          25.000000            13.000000            20.000000
75%              57.000000             972.000000             3.000000
1900.000000         38.000000            21.000000            30.000000
max              76.000000            3992.000000            20.000000
40425.000000        105.000000            105.000000            118.000000

```

keyfinding:

code:

```
merged = pd.merge(
    merged,
    order_items,
    on="order_id",
    how="inner"
)

merged.shape
```

output:

(112650, 18)

Code:

```
merged = pd.merge(
    merged,
    products,
    on="product_id",
    how="left"
)

print(merged.shape)
```

output:

(112650, 26)

Code:

```
print(merged["product_category_name"].value_counts().head(10))
```

output:

```
product_category_name
cama_mesa_banho      11115
beleza_saude          9670
esporte_lazer         8641
moveis_decoracao      8334
informatica_acessorios 7827
utilidades_domesticas 6964
relogios_presentes    5991
telefonica            4545
ferramentas_jardim    4347
automotivo            4235
```

keyfinding:

The analysis shows that **bed, bath, and table products** (`cama_mesa_banho`) are the most frequently purchased category in the dataset. Health & beauty and sports & leisure products also represent a significant share of customer purchases. These findings indicate that household and personal lifestyle products are the strongest product segments in this e-commerce platform.

The frequency analysis revealed that **Bed, Bath & Table** products were the most commonly purchased category, followed by **Health & Beauty** and **Sports & Leisure**. This suggests that household and lifestyle-related products represent the largest share of customer demand within the dataset.

The frequency analysis showed that the Bed, Bath & Table category was the most frequently ordered product category, followed by Health & Beauty and Sports & Leisure. This reflects customer purchasing preferences but does not necessarily indicate the highest revenue-generating categories.

After finding the answer of Which product categories are ordered most frequently? Now finding answer of Which product categories generate the highest revenue?

First looking dataset has price or not.

Code:

```
print(merged.columns)
```

output:

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',  
      'order_approved_at', 'order_delivered_carrier_date',  
      'order_delivered_customer_date', 'order_estimated_delivery_date',  
      'customer_unique_id', 'customer_zip_code_prefix', 'customer_city',  
      'customer_state', 'order_item_id', 'product_id', 'seller_id',  
      'shipping_limit_date', 'price', 'freight_value',  
      'product_category_name', 'product_name_lenght',  
      'product_description_lenght', 'product_photos_qty', 'product_weight_g',  
      'product_length_cm', 'product_height_cm', 'product_width_cm'],  
      dtype='object')
```

now knowing, have price

because of have price can write this code:

Code:

```
print(merged.groupby("product_category_name")["price"].sum().sort_values(ascending=False).head(10))
```

output:

```
product_category_name
beleza_saude          1258681.34
relogios_presentes    1205005.68
cama_mesa_banho       1036988.68
esporte_lazer         988048.97
informatica_acessorios 911954.32
moveis_decoracao      729762.49
cool_stuff            635290.85
utilidades_domesticas 632248.66
automotivo            592720.11
ferramentas_jardim    485256.46
Name: price, dtype: float64
```

Keyfinding:

The analysis revealed that Health & Beauty was the highest revenue-generating product category, with total sales revenue of 1,258,681.34, followed by Watches & Gifts (1,205,005.68) and Bed, Bath & Table (1,036,988.68).

Although Bed, Bath & Table was the most frequently ordered category in the previous analysis, Health & Beauty generated the highest revenue. This indicates that a higher number of orders does not necessarily result in the highest revenue. Therefore, both order frequency and revenue should be considered when evaluating product performance and making business decisions.

The analysis shows that Which product categories generate the highest revenue? Now time is for new analysis for finding answer of Which states generate the highest revenue?

Time conduct for geographic performance.

Code:

```
print(merged.groupby("customer_state")["price"] \
      .sum() \
      .sort_values(ascending=False) \
      .head(10)
)
```

Ourput:

```
customer_state
SP      5202955.05
RJ      1824092.67
MG      1585308.03
RS       750304.02
PR       683083.76
SC       520553.34
BA       511349.99
DF       302603.94
GO       294591.95
ES       275037.31
Name: price, dtype: float64
```

Keyfinding:

The analysis shows that **SP** generated the highest revenue (**5,202,955.05**), followed by **RJ** (**1,824,092.67**) and **MG** (**1,585,308.03**). This indicates that the company's revenue is highly concentrated in a few key states, making these regions the most important markets for the business. These states should be prioritized for marketing campaigns, customer retention initiatives, and future business expansion.

Code:

```
print(merged.columns)
```

output:

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date', 'order_estimated_delivery_date',
      'customer_unique_id', 'customer_zip_code_prefix', 'customer_city',
      'customer_state', 'order_item_id', 'product_id', 'seller_id',
      'shipping_limit_date', 'price', 'freight_value',
      'product_category_name', 'product_name_lenght',
      'product_description_lenght', 'product_photos_qty', 'product_weight_g',
      'product_length_cm', 'product_height_cm', 'product_width_cm'],
      dtype='object')
```

Which sellers generate the highest revenue?

Code:

```
print(merged.groupby("seller_id")["price"] \
```



```
.sum() \
.sort_values(ascending=False) \
.head(10)
)
```

Output:

```
seller_id
4869f7a5dfa277a7dca6462dcf3b52b2    229472.63
53243585a1d6dc2643021fd1853d8905    222776.05
4a3ca9315b744ce9f8e9374361493884    200472.92
fa1c13f2614d7b5c4749cbc52fecda94    194042.03
7c67e1448b00f6e969d365cea6b010ab    187923.89
7e93a43ef30c4f03f38b393420bc753a    176431.87
da8622b14eb17ae2831f4ac5b9dab84a    160236.57
7a67c85e85bb2ce8582c35f2203ad736    141745.53
1025f0e2d44d7041d6cf58b6550e0bfa    138968.55
955fee9216a65b617aa5c0531780ce60    135171.70
Name: price, dtype: float64
```

Keyfinding:

The analysis identifies the top 10 revenue-generating sellers based on total sales revenue. The results show that a small number of sellers contribute a significant share of the company's overall revenue. This indicates that these high-performing sellers play a critical role in the marketplace and should be prioritized for long-term partnerships, performance monitoring, and business growth strategies.

Which months generate the highest revenue?

Code:

```
merged.groupby("seller_id")["price"] \
.sum() \
.sort_values(ascending=False) \
.head(10)

merged["order_purchase_timestamp"] = pd.to_datetime(
    merged["order_purchase_timestamp"]
)

merged["purchase_month"] = merged["order_purchase_timestamp"].dt.month_name()
result=
merged.groupby("purchase_month")["price"].sum().sort_values(ascending=False)
print(result)
```

output:

```
purchase_month
May          1502588.82
August       1428658.01
July         1393538.70
March        1357557.74
April        1356574.98
June         1298162.91
February     1091481.73
January      1070343.23
November     1010271.37
December     743925.07
October      713727.09
September    624814.05
Name: price, dtype: float64
```

Keyfinding:

The analysis shows that May generated the highest revenue, followed by August and July. This indicates that customer purchasing activity was strongest during these months, resulting in higher sales revenue compared to the rest of the year. Understanding these seasonal trends can help the company optimize marketing campaigns, inventory planning, and promotional strategies during peak sales periods.

Now sell part is done and time is looking for delivery.

What is the average delivery time?

Code:

```
merged["order_delivered_customer_date"] = pd.to_datetime(
    merged["order_delivered_customer_date"]
)

merged["delivery_days"] = (
    merged["order_delivered_customer_date"]
    - merged["order_purchase_timestamp"]
).dt.days
merged["delivery_days"].mean()

print("Average:", merged["delivery_days"].mean())
```

```
print("Minimum:", merged["delivery_days"].min())
print("Maximum:", merged["delivery_days"].max())
```

output:

```
Average: 12.007722603361284
Minimum: 0.0
Maximum: 209.0
```

Keyfinding:

The analysis revealed that the average delivery time was approximately **12 days**, indicating that most customers received their orders within a relatively short period. However, delivery times ranged from **0 days** to **209 days**, suggesting that while some orders were delivered on the same day, others experienced exceptionally long delivery periods. These extreme delivery times may indicate operational delays or special order circumstances and should be investigated further to improve customer satisfaction.

What is the distribution of order statuses?

Code:

```
print(merged["order_status"].value_counts())
```

output:

```
order_status
delivered      110197
shipped         1185
canceled         542
invoiced         359
processing       357
unavailable         7
approved         3
```

keyfinding:

this code is be the raason of a mistake and must change the question.

How many orders are in each order status?

Code:

```
result =  
merged.groupby("order_status")["order_id"].nunique().sort_values(ascending=False)  
print(result)
```

output:

```
order_status  
delivered      96478  
shipped         1106  
canceled         461  
invoiced         312  
processing       301  
unavailable        6  
approved         2
```

keyfinding:

The analysis shows that **delivered** is by far the most common order status, with **96,478 unique orders**, followed by **shipped** with **1,106 orders**. This indicates that the vast majority of orders were successfully delivered to customers, suggesting a high overall order fulfillment rate. Only a relatively small number of orders were canceled, invoiced, processing, unavailable, or approved.

How accurate are the estimated delivery dates?

Code:

```
merged["order_estimated_delivery_date"] =pd.to_datetime(  
    merged["order_estimated_delivery_date"]  
)
```

```
merged["order_delivered_customer_date"] = pd.to_datetime(
    merged["order_delivered_customer_date"]
)

merged["delivery_difference_days"] = (
    merged["order_delivered_customer_date"]
    - merged["order_estimated_delivery_date"]
).dt.days

merged["delivery_difference_days"].mean()

print("average difference:" , merged["delivery_difference_days"].mean())
print("minimum difference:", merged["delivery_difference_days"].min())
print("maximum different:", merged["delivery_difference_days"].max())
```

output:

```
average difference: -12.030200733238956
minimum difference: -147.0
maximum different: 188.0
```

keyfinding:

The analysis shows that orders were delivered, on average, 12 days before the estimated delivery date. However, the delivery difference ranged from 147 days earlier to 188 days later than the estimated date, indicating substantial variation in delivery performance.

Which states have the longest average delivery times?

Code:

```
result= (
    merged.groupby("customer_state")["delivery_days"]
    .mean()
    .sort_values(ascending=False)
)

print(result.head(10))
```

output:

```
customer_state
RR      27.826087
AP      27.753086
```

```
AM    25.963190
AL    23.992974
PA    23.301708
MA    21.203750
SE    20.978667
CE    20.537167
AC    20.329670
PB    20.119454
```

Keyfinding:

The analysis shows that RR has the longest average delivery time at 27.83 days, followed by AP at 27.75 days and AM at 25.96 days. RR and AP have very similar average delivery times. This indicates that customers in these states experience longer delivery times compared with other regions, and these areas may require further investigation to identify potential logistics and delivery challenges.

How many unique customers placed orders?

Code:

```
print(merged["customer_unique_id"].nunique())
```

output:

```
95420
```

Keyfinding:

The analysis shows that 95,420 unique customers placed orders. This indicates that the company has a large customer base, with 95,420 unique customers purchasing through the platform.

How many customers made more than one purchase?

Code:

```
customer_orders =(
    merged.groupby("customer_unique_id")["order_id"]
    .nunique()
)

print(customer_orders.head(5))
```

output:

```
0      e481f51cbdc54678b7cc49136f2d6af7
1      53cdb2fc8bc7dce0b6741e2150273451
2      47770eb9100c2d0c44946d9cf07ec65d
3      949d5b44dbf5de918fe9c16f97b45f8a
4      ad21c59c0840e6cb83a9ceb5573f8159
...
112645 63943bddc261676b46f01ca7ac2f7bd8
112646 83c1379a015df1e13d02aae0204711ab
112647 11c177c8e97725db2631073c19f07b62
112648 11c177c8e97725db2631073c19f07b62
112649 66dea50a8b16d9b4dee7af250b4be1a5
```

Code:

```
repeat_customers=(customer_orders > 1).sum()
print("repeat customers:",repeat_customers)
```

output:

```
repeat customers: 2913
repeat_customers_rate = (
    repeat_customers / customer_orders.shape[0]
    *100
)
print("repeat_customers_rate:",repeat_customers_rate)
```

output:

```
repeat_customers_rate: 3.0528191154894153
```

The analysis shows that 2,913 customers made more than one purchase, representing approximately 3.05% of the total unique customer base. This indicates that the vast majority of customers made only one purchase, while a relatively small proportion returned to make additional purchases.

What is the average order value?

Code:

```
order_values = (
    merged.groupby("order_id")["price"]
    .sum()
)

print(order_values.head())

average_order_value = order_values.mean()
print("average order value:",average_order_value)

print("average order value:", order_values.mean())
print("minimum order value", order_values.min())
print("maximum order value:",order_values.max())
```

```

output:
order_id
00010242fe8c5a6d1ba2dd792cb16214      58.90
00018f77f2f0320c557190d7a144bdd3      239.90
000229ec398224ef6ca0657da4fc703e      199.00
00024acbcd0a6daa1e931b038114c75       12.99
00042b26cf59d7ce69dfabb4e55b4fd9      199.90
Name: price, dtype: float64
average order value: 137.75407637889444
average order value: 137.75407637889444
minimum order value 0.85
maximum order value: 13440.0

```

keyfinding:

The analysis shows that the average order value was approximately 137.75. Order values ranged from 0.85 to 13,440.00, indicating significant variation in customer spending per order. This information can help the company evaluate pricing strategies and identify opportunities to increase average order value.

Which states have the highest number of customers?

Code:

```

result = (
    merged.groupby("customer_state")["customer_unique_id"]
    .nunique()
    .sort_values(ascending=False)
)
print(result.head(10))

```

output:

```

order_id
00010242fe8c5a6d1ba2dd792cb16214      58.90
00018f77f2f0320c557190d7a144bdd3      239.90
000229ec398224ef6ca0657da4fc703e      199.00
00024acbcd0a6daa1e931b038114c75       12.99
00042b26cf59d7ce69dfabb4e55b4fd9      199.90
Name: price, dtype: float64
average order value: 137.75407637889444
customer_state
SP      39981

```


RJ	12303
MG	11178
RS	5249
PR	4840
SC	3513
BA	3257
DF	2062
ES	1956
GO	1942

Keyfinding:

The analysis shows that SP has the highest number of customers, with 39,981 unique customers, followed by RJ with 12,303 and MG with 11,178. This indicates that São Paulo is the company's largest customer market among the states in the dataset.

Which product categories are most popular in the largest customer states?

Code:

```
top_states = ["SP", "RJ", "MG"]

result = (
    merged[merged["customer_state"].isin(top_states)]
    .groupby(["customer_state", "product_category_name"])["order_id"]
    .nunique()
    .reset_index(name="order_count")
    .sort_values(
        ["customer_state", "order_count"],
        ascending=[True, False]
    )
)

#print(result.head(15))
print(result[result["customer_state"] == "SP"].head(10))
print(result[result["customer_state"] == "RJ"].head(10))
print(result[result["customer_state"] == "MG"].head(10))
```

output:

order_id	
00010242fe8c5a6d1ba2dd792cb16214	58.90
00018f77f2f0320c557190d7a144bdd3	239.90

```

000229ec398224ef6ca0657da4fc703e    199.00
00024acbcd0a6daa1e931b038114c75     12.99
00042b26cf59d7ce69dfabb4e55b4fd9    199.90
Name: price, dtype: float64
average order value: 137.75407637889444

```

	customer_state	product_category_name	order_count
153	SP	cama_mesa_banho	4416
151	SP	beleza_saude	3789
172	SP	esporte_lazer	3296
211	SP	utilidades_domesticas	2781
194	SP	moveis_decoracao	2724
184	SP	informatica_acessorios	2679
206	SP	relogios_presentes	2135
148	SP	automotivo	1615
152	SP	brinquedos	1612
209	SP	telefonica	1521

	customer_state	product_category_name	order_count
82	RJ	cama_mesa_banho	1393
80	RJ	beleza_saude	974
101	RJ	esporte_lazer	925
113	RJ	informatica_acessorios	859
122	RJ	moveis_decoracao	858
134	RJ	relogios_presentes	817
139	RJ	utilidades_domesticas	746
81	RJ	brinquedos	552
109	RJ	ferramentas_jardim	550
95	RJ	cool_stuff	503

	customer_state	product_category_name	order_count
13	MG	cama_mesa_banho	1129
11	MG	beleza_saude	1011
42	MG	informatica_acessorios	877
31	MG	esporte_lazer	867
52	MG	moveis_decoracao	727
69	MG	utilidades_domesticas	702
64	MG	relogios_presentes	611
38	MG	ferramentas_jardim	488
8	MG	automotivo	471
67	MG	telefonica	460

Keyfindig:

The analysis shows that `cama_mesa_banho` is the most frequently ordered product category in all three major customer states: SP, RJ, and MG. `beleza_saude` is the second most popular category across these states. This indicates a consistent customer preference for these categories across the company's largest markets. This insight can help the company prioritize inventory and marketing strategies for these categories in its major customer regions.

How does revenue change over time?

Code:

```
merged["purchase_date"] = pd.to_datetime(
    merged["order_purchase_timestamp"]
)

monthly_revenue = (
    merged.groupby(
        merged["purchase_date"].dt.to_period("M")
    )["price"]
    .sum()
)
print(monthly_revenue)
```

output:

```
order_id
00010242fe8c5a6d1ba2dd792cb16214    58.90
00018f77f2f0320c557190d7a144bdd3   239.90
000229ec398224ef6ca0657da4fc703e   199.00
00024acbcdff0a6daa1e931b038114c75    12.99
00042b26cf59d7ce69dfabb4e55b4fd9   199.90
Name: price, dtype: float64
average order value: 137.75407637889444
purchase_date
2016-09      267.36
2016-10    49507.66
2016-12      10.90
2017-01    120312.87
2017-02    247303.02
2017-03    374344.30
2017-04    359927.23
2017-05    506071.14
2017-06    433038.60
2017-07    498031.48
2017-08    573971.68
2017-09    624401.69
2017-10    664219.43
2017-11    1010271.37
2017-12    743914.17
2018-01    950030.36
2018-02    844178.71
2018-03    983213.44
2018-04    996647.75
2018-05    996517.68
2018-06    865124.31
2018-07    895507.22
2018-08    854686.33
2018-09    145.00
```

Keyfinding:

The analysis shows that revenue generally increased over time, rising from relatively low levels in early 2017 to much higher levels throughout 2018. Revenue reached its highest point in November 2017 at approximately 1.01 million, followed by consistently high monthly revenue in 2018. This indicates strong growth in sales activity over the analyzed period, although the sharp decline in September 2018 should be investigated further.

Which product categories generate high revenue but have relatively low order volume?

Code:

```
merged["purchase_date"] = pd.to_datetime(
    merged["order_purchase_timestamp"]
)

monthly_revenue = (
    merged.groupby(
        merged["purchase_date"].dt.to_period("M")
    )["price"]
    .sum()
)

print(monthly_revenue)
```

output:

```
purchase_date
2016-09         267.36
2016-10       49507.66
2016-12         10.90
2017-01      120312.87
2017-02      247303.02
2017-03      374344.30
2017-04      359927.23
2017-05      506071.14
2017-06      433038.60
2017-07      498031.48
2017-08      573971.68
2017-09      624401.69
2017-10      664219.43
2017-11     1010271.37
2017-12      743914.17
2018-01      950030.36
2018-02      844178.71
2018-03      983213.44
```

```

2018-04      996647.75
2018-05      996517.68
2018-06      865124.31
2018-07      895507.22
2018-08      854686.33
2018-09         145.00
Freq: M, Name: price, dtype: float64

```

Which product categories generate the highest revenue per order?

Code:

```

category_analysis = (
    merged.groupby("product_category_name")
    .agg(
        total_revenue=("price", "sum"),
        order_count=("order_id", "nunique")
    )
)
category_analysis["revenue_per_order"] = (
    category_analysis["total_revenue"]
    / category_analysis["order_count"]
)
result = category_analysis.sort_values(
    "revenue_per_order",
    ascending=False
)
print(result.head(10))

```

output:

```

revenue_per_order      total_revenue  ...
product_category_name      ...
pcs                    222963.13  ...
1231.840497
portateis_casa_forno_e_cafe    47445.71  ...
632.609467
eletrodomesticos_2          113317.74  ...
484.263846
agro_industria_e_comercio      72530.47  ...
398.519066
instrumentos_musicais          191498.88  ...
304.934522

```

```
eletroportateis          190648.58  ...
302.616794
portateis_cozinha_e_preparadores_de_alimentos  3968.53  ...
283.466429
telefonica_fixa          59583.00  ...
274.576037
construcao_ferramentas_seguranca  40544.52  ...
242.781557
climatizacao            55024.96  ...
217.489960

[10 rows x 3 columns]
```

Keyfinding:

The analysis shows that the `pcs` category generates the highest revenue per order, with an average of approximately 1,231.84 per order. It is followed by `portateis_casa_forno_e_cafe` at 632.61 and `eletrodomesticos_2` at 484.26. This indicates that some product categories generate relatively high revenue from each order even when their total revenue is not among the highest. These categories may therefore represent valuable opportunities for the company to focus on high-value sales.

Conclusion

This project provided an opportunity to analyze an e-commerce dataset using Python and explore the business performance of the company.

During the analysis, different aspects of the business were examined, including revenue, orders, customers, product categories, sellers, customer states, order statuses, and delivery performance. These analyses helped identify important patterns in the dataset and understand the factors that affect the company's sales and customer activity.

One of the main findings was that some product categories and customer states generated significantly higher revenue than others. The analysis also showed that the majority of orders were successfully delivered and that, on average, orders were delivered before the estimated delivery date.

Overall, this project improved my understanding of data cleaning, exploratory data analysis, business questions, and interpreting data with Python. It also helped me practice turning raw data into meaningful business insights.

The next step in developing this project could be building an interactive dashboard and extending the analysis with additional business questions.